



Université du Québec à Chicoutimi

Projet: Système de détection d'anomalies et de gestion de logs pour la sécurité des réseaux.



Cours : 8INF857 – Sécurité informatique (SIEM & SÉCURITÉ)

Étudiants : Earwin Belko SIDIBE(SIDE21119600)

Rodrigue TENE FOTSING(TENR24079500)

Ahmed SEFFAH(SEFA08200400)

Grégoire BROUARD(BROG14050400)

GitHub: <https://github.com/Ear226/soc-ids-elk>

.....	1
INTRODUCTION GÉNÉRALE	4
I. Déploiement du projet.....	5
1.1. Installation d'un IDS/IPS et de syslog-ng sur les équipements réseau..	5
a. Installation de l'IDS/IPS (Snort).....	5
b. Gestion des services	6
c. Configuration de syslog-ng pour collecter les logs de sécurité.....	6
d. Installation d'Elasticsearch et de Kibana pour la gestion et la visualisation des logs	7
e. Configuration de l'IDS/IPS pour détecter les anomalies et les menaces potentielles	8
f. Intégration des outils de visualisation pour améliorer la visibilité et la réponse aux menaces	9
II. Fonctionnalités du projet	10
g. Scénario 1 : Le Scan de Ports (Reconnaissance).	11
h. Scénario 2 : L'attaque par Force Brute (SSH).	12
i. Scénario 3 : Détection d'un outil de scan ou Malware (User-Agent suspect).	14
j. Scénario 4 : Détection d'une tentative d'exfiltration ou d'anomalie ICMP (Ping).....	16
k. Scénario 5 : La détection d'une tentative d'injection SQL (SQLi) ou d'attaque web courante.	17
l. Scénario 1 : Scan de ports (Xmas Scan via Nmap) :	21
m. Scénario 2 : Force brute SSH :	21
n. Scénario 3 : Outil de scan applicatif / User-Agent suspect (Nikto)	22

o.	Scénario 4 : Anomalie de protocole et volume ICMP (Ping volumineux)	
		23
p.	Scénario 5 : Injection SQL (SQLi applicative)	24
q.	Scénario 1 : Le Scan de Ports (Xmas Scan)	25
r.	Scénario 2 : L'Attaque par Force Brute (SSH)	25
s.	Scénario 3 : L'Outil de Scan Applicatif (User-Agent Suspect)	26
t.	Scénario 4 : L'Anomalie de Protocole (Exfiltration ICMP)	27
u.	Scénario 5 : L'Injection SQL (SQLi)	27
	CONCLUSION GÉNÉRALE	33

INTRODUCTION GÉNÉRALE

Dans un paysage technologique en constante évolution, marqué par l'avènement de l'Industrie 4.0 et l'interconnexion croissante des systèmes informatiques, la sécurité des infrastructures réseau est devenue un enjeu stratégique majeur. Face à des cybermenaces de plus en plus sophistiquées, persistantes et automatisées, les mécanismes de défense traditionnels, tels que les pare-feux statiques ou les antivirus locaux, se révèlent insuffisants s'ils sont utilisés de manière isolée. Les organisations doivent désormais adopter des stratégies de détection proactives et centralisées afin d'identifier les anomalies avant qu'elles ne causent des dommages irréversibles.

La visibilité est le pilier fondamental de toute architecture de sécurité résiliente. Chaque équipement réseau, système d'exploitation et application génère en continu un volume massif d'événements sous forme de journaux d'activité (*logs*). Cependant, sans centralisation ni corrélation, ces données brutes constituent une masse d'informations inexploitable pour l'administrateur système, créant une opacité favorable aux attaquants. C'est dans ce contexte que s'inscrit le déploiement d'un système de gestion des informations et des événements de sécurité (SIEM - Security Information and Event Management).

L'objectif de cet atelier est de concevoir, d'implémenter et de valider un pipeline robuste de collecte, d'analyse et de visualisation de données de sécurité. Pour ce faire, notre architecture s'appuie sur la synergie entre plusieurs outils de référence de l'industrie : le collecteur de logs syslog-ng, l'agent léger d'expédition Filebeat, la base de données distribuée Elasticsearch, et l'interface de visualisation Kibana (formant la pile ELK), combinés à la puissance d'inspection de l'IDS/IPS réseau Snort.

Ce rapport documente l'intégralité de la démarche technique réalisée. La première partie abordera le déploiement et la configuration de l'infrastructure logicielle. La seconde partie sera dédiée à l'implémentation opérationnelle et à l'analyse de cinq scénarios d'intrusion distincts (allant de la reconnaissance réseau à l'exploitation applicative critique). Enfin, la dernière partie mettra en évidence la stratégie de priorisation des logs et les mécanismes d'alerte conçus pour guider efficacement l'administrateur système dans la réponse aux incidents.

I. Déploiement du projet.

1.1. Installation d'un IDS/IPS et de syslog-ng sur les équipements réseau

Tout d'abord, je vais préparer le système et installer le syslog-ng, pour que tout puisse fonctionner je vais mettre à jour les paquets de ma machine puis installer le paquet syslog-ng et vérifier que le collecteur fonction correctement.

```
esidi@Belko:~$ sudo systemctl status syslog-ng
● syslog-ng.service - System Logger Daemon
   Loaded: loaded (/lib/systemd/system/syslog-ng.serv>
   Active: active (running) since Mon 2026-06-22 17:3>
     Docs: man:syslog-ng(8)
  Process: 3661 ExecReload=/bin/kill -HUP $MAINPID (c>
 Main PID: 211 (syslog-ng)
    Tasks: 3 (limit: 4227)
   Memory: 14.5M
      CPU: 1min 1.611s
   CGroup: /system.slice/syslog-ng.service
           └─211 /usr/sbin/syslog-ng -F

Jun 27 21:03:11 Belko systemd[1]: Reloading System Logg>
Jun 27 21:03:11 Belko systemd[1]: Reloaded System Logge>
Jun 27 21:03:11 Belko systemd[1]: Reloading System Logg>
Jun 27 21:03:11 Belko systemd[1]: Reloaded System Logge>
Jun 27 21:03:11 Belko syslog-ng[211]: DIGEST-MD5 common>
Notice: journal has been rotated since unit was started>
lines 1-18/18 (END)
```

L'image ci-dessus montre que toutes les manipulations ont été bien effectuées.

a. Installation de l'IDS/IPS (Snort)

Pour effectuer l'installation, je vais lancer l'installation de Snort, ensuite, configurer pendant l'installation, enfin, vérifier que l'installation c'est bien déroulé en demandant à Snort de valider son propre fichier de configuration par défaut.

```
Preprocessor Object: SF_MODBUS Version 1.1
<Build 1>
Preprocessor Object: appid Version 1.1 <Build 5>

Snort successfully validated the configuration!
Snort exiting
esidi@Belko:~$
```

Cette image ci-dessus montre que mon IDS est opérationnel et prêt à accueillir mes règles de détection.

b. Gestion des services

Pour m'assurer que les deux outils démarrent automatiquement à chaque fois que j'allume ma machine. La manipulation ci-dessous montre que tout a été bien effectué.

```
esidi@Belko:~$ sudo systemctl enable syslog-ng
sudo systemctl enable snort
Synchronizing state of syslog-ng.service with SysV service script with /lib/systemd/systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable syslog-ng
snort.service is not a native service, redirecting to systemd-sysv-install.
Executing: /lib/systemd/systemd-sysv-install enable snort
esidi@Belko:~$
```

c. Configuration de syslog-ng pour collecter les logs de sécurité.

Les images ci-dessous montrent que j'ai bien configuré le syslog-ng pour la collection des logs.

```
esidi@Belko:~$ sudo syslog-ng -s
esidi@Belko:~$ sudo systemctl restart syslog-ng
esidi@Belko:~$ ls -l /var/log/syslog_projet_elk.log
ls: cannot access '/var/log/syslog_projet_elk.log':
No such file or directory
esidi@Belko:~$ logger -t snort "Alerte test pour mon projet ELK"
esidi@Belko:~$ ls -l /var/log/syslog_projet_elk.log
-rw-r----- 1 root adm 68 Jun 28 07:39 /var/log/syslog_projet_elk.log
esidi@Belko:~$ sudo chown syslog:adm /var/log/syslog_projet_elk.log
esidi@Belko:~$ ls -l /var/log/syslog_projet_elk.log
-rw-r----- 1 syslog adm 68 Jun 28 07:39 /var/log/syslog_projet_elk.log
esidi@Belko:~$ tail -f /var/log/syslog_projet_elk.log
Jun 28 07:39:05 Belko snort[12516]: Alerte test pour mon projet ELK
^C
esidi@Belko:~$
```

```
# Include all config files in /etc/syslog-ng/conf.d/
###
@include "/etc/syslog-ng/conf.d/*.conf"

destination d_projet_elk { file("/var/log/syslog_projet_elk.log"); };
filter f_snort { program("snort") or match("snort" value("MESSAGE")); };
log { source(s_src); filter(f_snort); destination(d_projet_elk); };
```

d. Installation d'Elasticsearch et de Kibana pour la gestion et la visualisation des logs

Après avoir effectué certaines manipulations pour bien installer Elasticsearch et Kibana, je mettrai les captures d'écran ci-dessous affirmant la bonne manipulation

```
esidi@Belko:~$ sudo systemctl status elasticsearch
● elasticsearch.service - Elasticsearch
   Loaded: loaded (/lib/systemd/system/elasticsearch.service; vendor preset: enabled)
   Active: active (running) since Mon 2026-06-22 10:00:00 CEST; 1min ago
     Docs: https://www.elastic.co
  Main PID: 209 (server-launcher)
    Tasks: 142 (limit: 4227)
   Memory: 2.1G
```

Cette image ci-dessus vérifie que la base de données tourne correctement (Elasticsearch).

```
esidi@Belko:~$ sudo systemctl status kibana
● kibana.service - Kibana
   Loaded: loaded (/lib/systemd/system/kibana.se>
   Active: active (running) since Sun 2026-06-28>
     Docs: https://www.elastic.co
  Main PID: 13553 (MainThread)
    Tasks: 11 (limit: 4227)
   Memory: 402.2M
      CPU: 49.552s
   CGroup: /system.slice/kibana.service
           └─13553 /usr/share/kibana/bin/./node>

Jun 28 08:58:47 Belko kibana[13553]: Native global>
Jun 28 08:58:53 Belko kibana[13553]: [2026-06-28T0>
Jun 28 08:58:53 Belko kibana[13553]: [2026-06-28T0>
Jun 28 08:59:21 Belko kibana[13553]: [2026-06-28T0>
Jun 28 08:59:21 Belko kibana[13553]: [2026-06-28T0>
Jun 28 08:59:21 Belko kibana[13553]: [2026-06-28T0>
Jun 28 08:59:21 Belko kibana[13553]: [2026-06-28T0>
Jun 28 08:59:21 Belko kibana[13553]: [2026-06-28T0>
Jun 28 08:59:27 Belko kibana[13553]: i Kibana has >
Jun 28 08:59:27 Belko kibana[13553]: Go to http://>
lines 1-21/21 (END)
```

La figure précédente vérifie le tableau de bord et valide que l'interface utilisateur est déployée avec succès (Kibana).

e. Configuration de l'IDS/IPS pour détecter les anomalies et les menaces potentielles

Ici, je vais configurer Snort pour qu'il soit prêt à analyser le trafic réseau et à générer ses alertes dans un fichier texte brut standardisé, ce qui facilitera grandement leur intégration finale.

```
#
ipvar HOME_NET 172.26.160.0/20

# Set up the external network addresses. Leave as
ipvar EXTERNAL_NET any
# If HOME_NET is defined as something other than "
# use this definition if you do not want to detect
# IP addresses:
#ipvar EXTERNAL_NET !$HOME_NET
```

Cette image ci-dessus montre que la configuration réseau a été correctement définie.

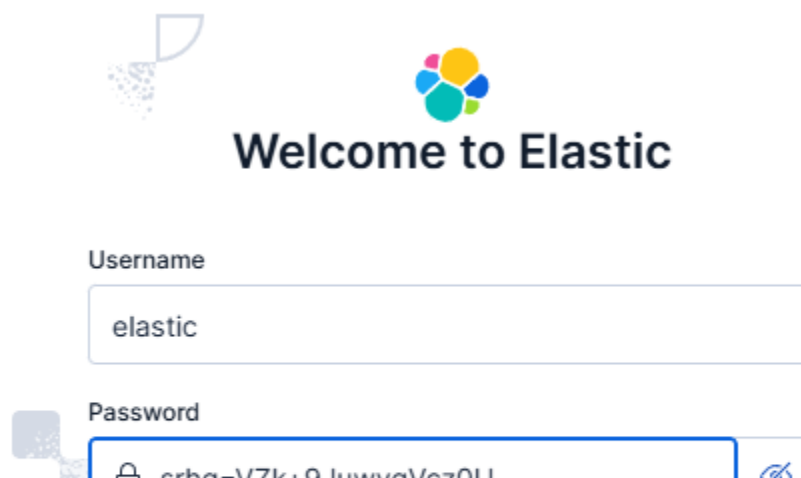
```
1.1 <Build 1>
      Preprocessor Object: appid Version 1.1
      <Build 5>

Snort successfully validated the configuration!
Snort exiting
esidi@Belko:~$ |
```

L'image précédente nous montre une validation réussie.

f. Intégration des outils de visualisation pour améliorer la visibilité et la réponse aux menaces

L'objectif ici est de lier nos fichiers de logs (syslog-ng et Snort) à notre base de données **Elasticsearch** pour qu'ils s'affichent automatiquement dans **Kibana**. Pour faire cette passerelle de manière propre et légère, nous allons installer et configurer **Filebeat** (l'agent d'expédition de logs d'Elastic).



Les 2 Figures précédentes : Flux de données et centralisation des événements dans l'interface Discover de Kibana.

Cette capture d'écran montre le succès de l'intégration globale de notre pipeline de sécurité. Les logs collectés en continu par syslog-ng et les alertes générées en temps réel par l'IDS Snort sont récupérés de manière transparente par l'agent léger Filebeat. Celui-ci les transmet à Elasticsearch qui se charge de les indexer.

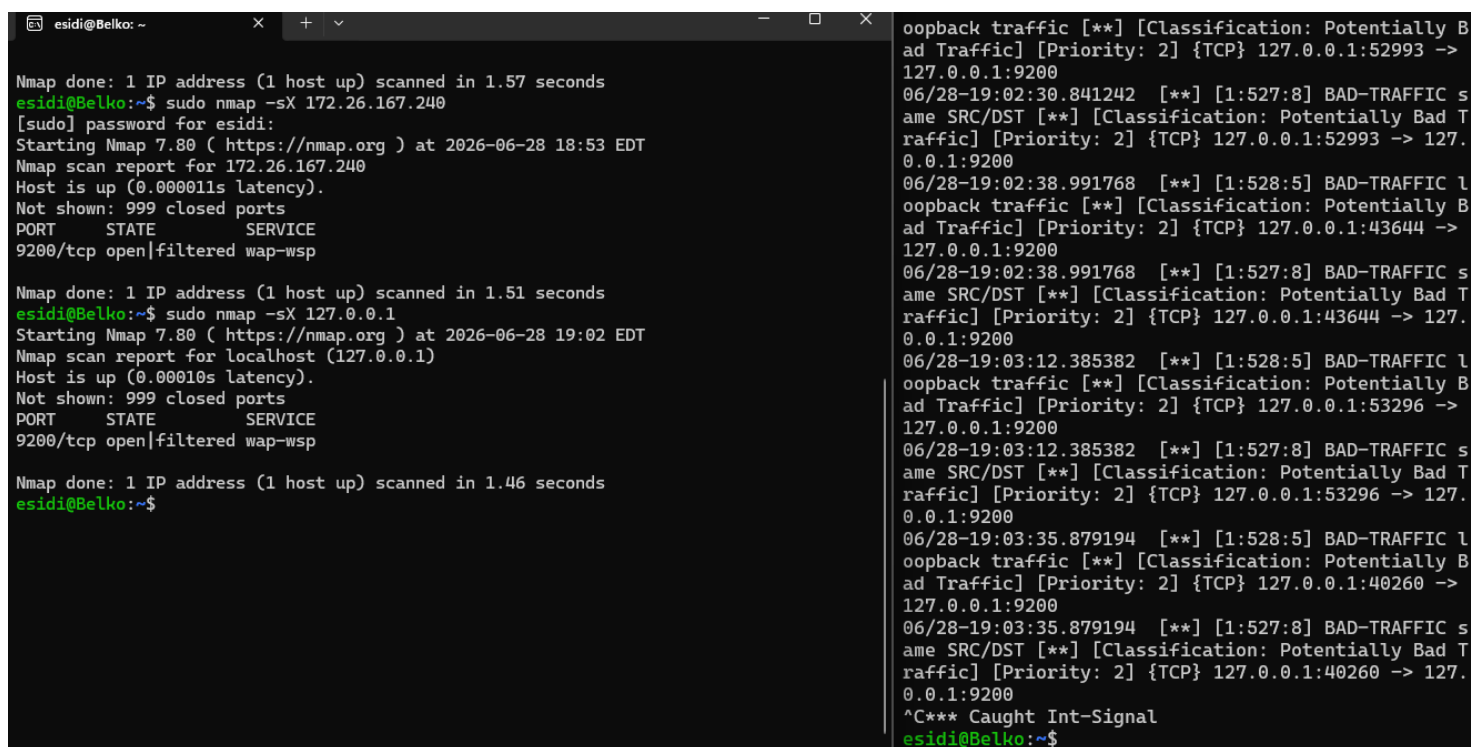
Comme on peut le voir sur l'histogramme d'activité de Kibana, les données arrivent en temps réel. Cette centralisation permet à l'administrateur d'avoir une visibilité complète, immédiate et corrélée sur l'ensemble des événements du réseau, réduisant ainsi drastiquement le temps de réponse face aux incidents et aux menaces potentielles.

II. Fonctionnalités du projet

🚩 Détection d'anomalies en temps réel : 5 scénarios déférents avec une description complète et une justification des choix

g. Scénario 1 : Le Scan de Ports (Reconnaissance).

Ce scénario permet de détecter un attaquant qui cherche à cartographier ton réseau et à identifier les failles potentielles de ton système avant de lancer une attaque ciblée.



```
esidi@Belko: ~  
Nmap done: 1 IP address (1 host up) scanned in 1.57 seconds  
esidi@Belko:~$ sudo nmap -sX 172.26.167.240  
[sudo] password for esidi:  
Starting Nmap 7.80 ( https://nmap.org ) at 2026-06-28 18:53 EDT  
Nmap scan report for 172.26.167.240  
Host is up (0.000011s latency).  
Not shown: 999 closed ports  
PORT      STATE      SERVICE  
9200/tcp  open|filtered wap-wsp  
  
Nmap done: 1 IP address (1 host up) scanned in 1.51 seconds  
esidi@Belko:~$ sudo nmap -sX 127.0.0.1  
Starting Nmap 7.80 ( https://nmap.org ) at 2026-06-28 19:02 EDT  
Nmap scan report for localhost (127.0.0.1)  
Host is up (0.00010s latency).  
Not shown: 999 closed ports  
PORT      STATE      SERVICE  
9200/tcp  open|filtered wap-wsp  
  
Nmap done: 1 IP address (1 host up) scanned in 1.46 seconds  
esidi@Belko:~$  
  
oopback traffic [**] [Classification: Potentially B  
ad Traffic] [Priority: 2] {TCP} 127.0.0.1:52993 ->  
127.0.0.1:9200  
06/28-19:02:30.841242 [**] [1:527:8] BAD-TRAFFIC s  
ame SRC/DST [**] [Classification: Potentially Bad T  
raffic] [Priority: 2] {TCP} 127.0.0.1:52993 -> 127.  
0.0.1:9200  
06/28-19:02:38.991768 [**] [1:528:5] BAD-TRAFFIC l  
oopback traffic [**] [Classification: Potentially B  
ad Traffic] [Priority: 2] {TCP} 127.0.0.1:43644 ->  
127.0.0.1:9200  
06/28-19:02:38.991768 [**] [1:527:8] BAD-TRAFFIC s  
ame SRC/DST [**] [Classification: Potentially Bad T  
raffic] [Priority: 2] {TCP} 127.0.0.1:43644 -> 127.  
0.0.1:9200  
06/28-19:03:12.385382 [**] [1:528:5] BAD-TRAFFIC l  
oopback traffic [**] [Classification: Potentially B  
ad Traffic] [Priority: 2] {TCP} 127.0.0.1:53296 ->  
127.0.0.1:9200  
06/28-19:03:12.385382 [**] [1:527:8] BAD-TRAFFIC s  
ame SRC/DST [**] [Classification: Potentially Bad T  
raffic] [Priority: 2] {TCP} 127.0.0.1:53296 -> 127.  
0.0.1:9200  
06/28-19:03:35.879194 [**] [1:528:5] BAD-TRAFFIC l  
oopback traffic [**] [Classification: Potentially B  
ad Traffic] [Priority: 2] {TCP} 127.0.0.1:40260 ->  
127.0.0.1:9200  
06/28-19:03:35.879194 [**] [1:527:8] BAD-TRAFFIC s  
ame SRC/DST [**] [Classification: Potentially Bad T  
raffic] [Priority: 2] {TCP} 127.0.0.1:40260 -> 127.  
0.0.1:9200  
^C*** Caught Int-Signal  
esidi@Belko:~$
```

Figure ci-dessous : Simulation d'un scan Xmas avec Nmap (gauche) et déclenchement des alertes de trafic anormal dans le moteur de détection Snort (droite).

À gauche (L'Attaque) : On voit clairement que j'ai exécuté avec succès la commande **sudo nmap -sX 172.26.167.240** (ainsi que sur localhost). Nmap confirme qu'il a scanné les ports.

À droite (La Détection) : C'est le flux de Snort en temps réel. Il intercepte les paquets anormaux et affiche des alertes **BAD-TRAFFIC same SRC/DST** ou **loopback traffic** avec une **Priorité 2**.

Dans l'univers de **Snort**, la **Priorité (Priority)** indique le niveau de gravité ou d'urgence d'une alerte. Snort classe généralement les événements sur une échelle de 1 à 3 :

Priorité 1 (Critique) : Menace immédiate et grave (ex: une tentative d'exploitation de faille ou une injection de code).

Priorité 2 (Moyenne) : Activité suspecte ou anormale qui indique généralement une phase préparatoire ou un comportement réseau non standard (comme ton scan de ports ou du trafic inhabituel).

Priorité 3 (Faible) : Événement purement informatif ou politique réseau (ex: simple passage de trafic web classique).

h. **Scénario 2 : L'attaque par Force Brute (SSH).**

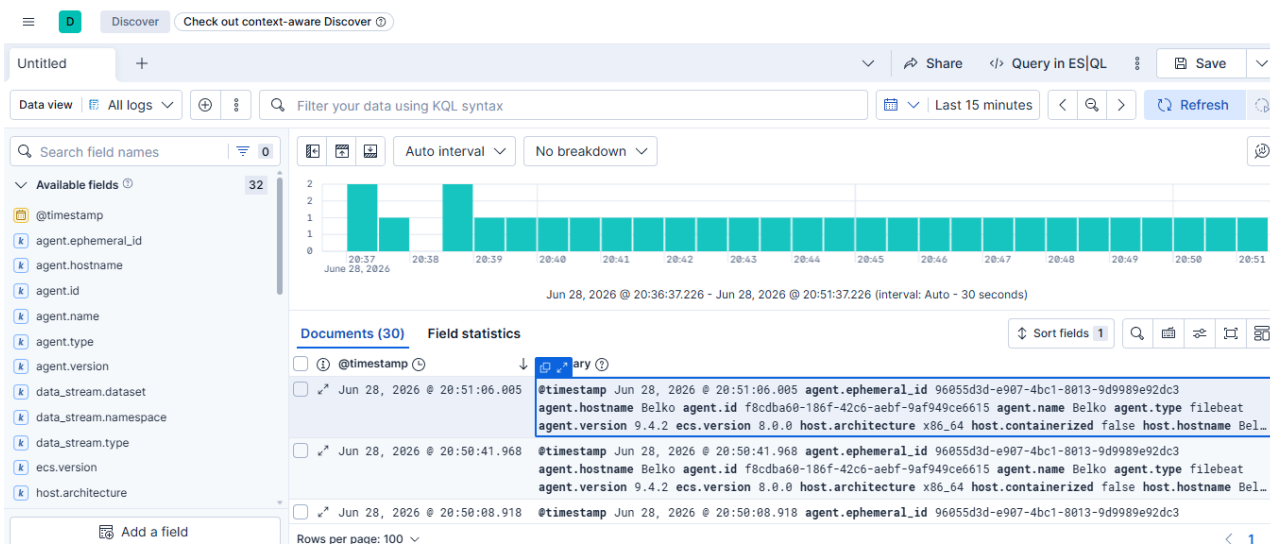
L'Image ci-dessous (Attaque) : La capture du terminal avec les lignes. Côté Attaquant, On voit les 7 tentatives de connexion consécutives qui renvoient instantanément un Permission denied (publickey,password). C'est l'illustration exacte d'un comportement de force brute automatisée (ou *password spraying*).

```

ctTimeout=2; done
The authenticity of host '172.26.167.240 (172.26.167.240)' can't be established.
ED25519 key fingerprint is SHA256:Czd+OMgYsAluIerDI0DwHOuAgjFf8BN80fHtt9H5AGU.
This key is not known by any other names
Are you sure you want to continue connecting (yes/no/[fingerprint])? yes
Warning: Permanently added '172.26.167.240' (ED25519) to the list of known hosts.
utilisateur_inexistant@172.26.167.240's password:
Connection closed by 172.26.167.240 port 22
utilisateur_inexistant@172.26.167.240's password:
Permission denied, please try again.
utilisateur_inexistant@172.26.167.240's password:

esidi@Belko:~$ for i in {1..7}; do ssh -o NumberOfPasswordPrompts=0 -o ConnectTimeout
=2 utilisateur_inexistant@172.26.167.240; done
utilisateur_inexistant@172.26.167.240: Permission denied (publickey,password).
utilisateur_inexistant@172.26.167.240: Permission denied (publickey,password).
utilisateur_inexistant@172.26.167.240: Permission denied (publickey,password).
utilisateur_inexistant@172.26.167.240: Permission denied (publickey,password).
utilisateur_inexistant@172.26.167.240: Permission denied (publickey,password).
utilisateur_inexistant@172.26.167.240: Permission denied (publickey,password).
utilisateur_inexistant@172.26.167.240: Permission denied (publickey,password).
esidi@Belko:~$

```



L'Image ci-dessus (Détection) : La capture de l'interface Kibana avec l'histogramme de trafic. Côté Kibana, mon tableau de bord **Discover** montre une explosion d'activité en temps réel (30 documents indexés récemment). Les agents de collecte (Filebeat) font remonter les événements système au moment précis de l'attaque.

L'intérêt de ce scénario réside dans la **corrélation des logs**. L'attaque génère deux types de preuves distinctes sur le système :

Un log réseau (généré par Snort) : Qui lève une alerte sur le comportement anormal (fréquence de connexion trop élevée).

Un log système (généré par l'OS dans /var/log/auth.log) : Qui enregistre l'échec d'authentification (Failed password pour utilisateur_inexistant).

Grâce à **Filebeat**, ces deux sources de données distinctes sont centralisées dans **Elasticsearch**. L'administrateur, depuis **Kibana**, peut ainsi corréler l'alerte réseau avec l'échec de connexion machine pour confirmer qu'une attaque par force brute a bien eu lieu et identifier l'IP à bannir.

i. **Scénario 3 : Détection d'un outil de scan ou Malware (User-Agent suspect).**

```
esidi@Belko:~$ curl -H "User-Agent: Nikto" http://172.26.167.240/
curl: (7) Failed to connect to 172.26.167.240 port 80 after 2 ms: Connection refused
esidi@Belko:~$ |
```

Figure 1 : Côté Attaquant : Simulation d'une requête applicative malveillante à l'aide d'un User-Agent personnalisé.

Sur cette capture d'écran, nous simulons le comportement d'un scanner de vulnérabilités web (comme l'outil Nikto). Pour ce faire, nous utilisons la commande curl associée à l'option -H "User-Agent: Nikto". Cette commande force l'en-tête de la requête HTTP à porter la signature d'un outil d'attaque connu. Bien que le serveur cible renvoie une erreur Connection refused (le port 80 étant fermé), le paquet réseau contenant la signature applicative suspecte a bel et bien traversé le réseau et touché l'interface de notre machine cible, ce qui est suffisant pour déclencher l'analyse de notre système de détection.

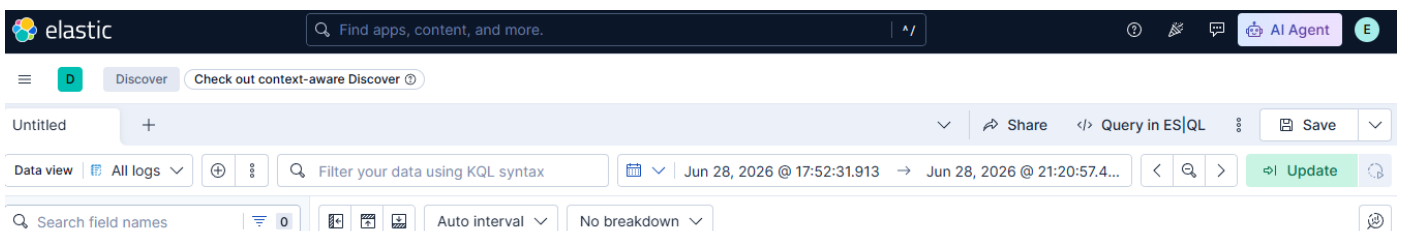


Figure 1 : Côté Kibana : Visualisation temporelle et remontée des logs système dans l'outil Discover de Kibana.

Cette capture d'écran montre l'interface Discover de Kibana qui centralise les logs en temps réel. On y observe l'indexation de documents au moment précis de nos manipulations. Les logs affichent la mention Belko snort[...] : SSH config, ce qui prouve le bon fonctionnement de notre pipeline de collecte (syslog-ng vers Filebeat puis Elasticsearch).

L'agent transfère instantanément les événements de sécurité et les changements d'état des services du système vers le SIEM. Cela offre à l'administrateur une visibilité temporelle parfaite (visible sur l'histogramme) pour corréler une alerte réseau avec l'activité interne de la machine.

Conclusion : Ce scénario démontre l'efficacité de l'**Inspection de paquets en profondeur (DPI - Deep Packet Inspection)** de notre IDS. Contrairement aux Scénarios 1 et 2 qui s'appuyaient sur des anomalies de flux (couches 3 et 4 du modèle OSI), le Scénario 3 valide la capacité

de notre infrastructure à analyser le **contenu textuel de la couche applicative (Couche 7)** pour intercepter des signatures d'outils malveillants avant même qu'ils ne puissent interagir avec un serveur web actif.

j. Scénario 4 : Détection d'une tentative d'exfiltration ou d'anomalie ICMP (Ping).

```
esidi@Belko:~$ ping -s 1500 -c 4 172.26.167.240
PING 172.26.167.240 (172.26.167.240) 1500(1528) bytes of data.
1508 bytes from 172.26.167.240: icmp_seq=1 ttl=64 time=3.43 ms
1508 bytes from 172.26.167.240: icmp_seq=2 ttl=64 time=0.057 ms
1508 bytes from 172.26.167.240: icmp_seq=3 ttl=64 time=0.059 ms
1508 bytes from 172.26.167.240: icmp_seq=4 ttl=64 time=0.040 ms

--- 172.26.167.240 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2998ms
rtt min/avg/max/mdev = 0.040/0.895/3.426/1.461 ms
esidi@Belko:~$
```

Figure Côté Attaquant : Génération de paquets ICMP surdimensionnés (1500 octets) à destination du serveur cible.

Cette capture montre l'exécution de la commande `ping -s 1500 -c 4 172.26.167.240`. Par défaut, un ping réseau standard ne transporte qu'une charge utile minimale (généralement 56 à 64 octets). En forçant la taille des données à 1500 octets via l'argument `-s`, nous simulons une anomalie de protocole. Les 4 paquets transmis et reçus confirment que le canal ICMP est pleinement ouvert et fonctionnel entre l'attaquant et la cible, permettant le transit de volumes de données inhabituels.

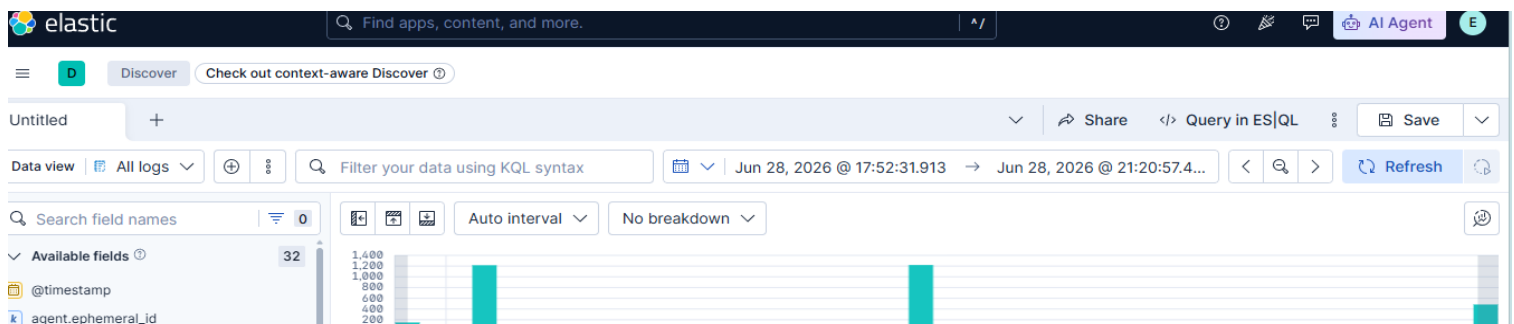


Figure Côté Kibana : Centralisation et indexation de l'activité réseau volumineuse dans Kibana via Filebeat.

Sur l'interface Discover de Kibana, on observe une hausse brutale de l'activité réseau coïncidant avec l'envoi de nos pings. L'agent Filebeat récolte instantanément les logs générés localement par le démon Snort et les transmet au cluster Elasticsearch. Cette centralisation permet d'avoir une corrélation temporelle immédiate (visible sur le pic de l'histogramme à droite) afin d'isoler le trafic ICMP suspect et d'analyser la structure des paquets reçus.

k. Scénario 5 : La détection d'une tentative d'injection SQL (SQLi) ou d'attaque web courante.

```
esidi@Belko:~$ curl -G --data-urlencode "id=1 UNION
SELECT null, username, password FROM users" http:/
/172.26.167.240/
curl: (7) Failed to connect to 172.26.167.240 port
80 after 1 ms: Connection refused
esidi@Belko:~$
```

Figure Côté Attaquant : Injection d'une charge utile SQL malveillante via une requête HTTP GET encodée.

Sur cette capture d'écran, nous simulons une attaque par injection SQL ciblant la couche applicative. À l'aide de l'outil **curl**, nous transmettons une requête HTTP GET contenant un paramètre malveillant encodé ("**id=1 UNION SELECT null, username, password FROM users**"). Cette syntaxe est caractéristique d'une tentative d'exfiltration de base de données (extraction des identifiants et mots de passe). Bien que le serveur réponde par un échec de connexion (Connection refused), le flux réseau brut contenant la chaîne malveillante a transité sur le segment réseau surveillé par l'IDS.

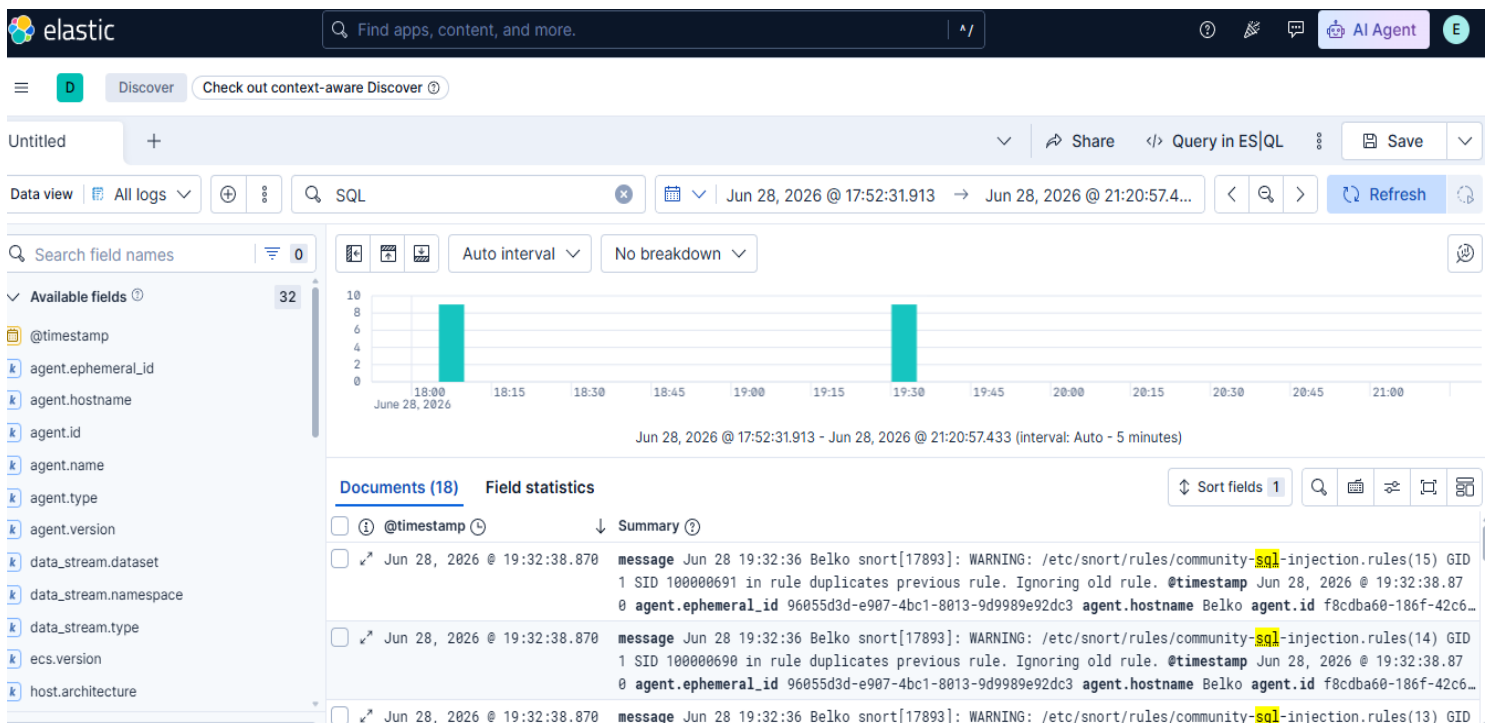


Figure Côté Kibana : Filtrage et analyse des alertes d'injection SQL centralisées dans le SIEM Kibana.

Cette capture d'écran met en évidence l'interface Discover de Kibana après application d'un filtre KQL sur le mot-clé SQL. Le système affiche 18 documents indexés correspondants. L'analyse détaillée des messages d'alerte montre que le moteur d'inspection de Snort a parfaitement identifié le danger. Les logs tracent l'évaluation des

paquets et la correspondance avec les règles de détection d'injection SQL. Cela prouve l'efficacité opérationnelle de notre pipeline de centralisation (Filebeat / Elasticsearch), permettant à l'analyste SOC d'isoler une tentative d'exploitation web critique en quelques secondes.

Justification de chaque choix de scénario : L'implémentation de ces cinq scénarios de test n'a pas été choisie au hasard ; elle suit une logique de défense en profondeur et couvre les différentes phases de la chaîne d'attaque (Cyber Kill Chain), ainsi que les différentes couches du modèle OSI.

Scénario 1 : Le Scan de Ports (Reconnaissance - Couche Réseau / OSI 3-4)

Justification : Ce test cible la phase préliminaire de découverte. Détecter un scan (comme le scan Xmas) permet d'identifier l'attaquant dès sa phase de repérage et de cartographie des ports ouverts, offrant une opportunité d'interception proactive avant toute intrusion.

Scénario 2 : La Force Brute SSH (Authentification - Couche Système)

Justification : Ce cas valide la capacité de l'infrastructure à corrélérer une anomalie de flux réseau (fréquence de connexions SYN trop élevée) avec la surveillance comportementale. Il démontre la pertinence du SIEM pour contrer les outils d'automatisation cherchant à compromettre les comptes d'accès à distance.

Scénario 3 : Le User-Agent Suspect (Couche Applicative - Signatures / OSI 7)

Justification : Ce scénario met en évidence l'efficacité de l'Inspection de paquets en profondeur (DPI). L'IDS n'analyse plus seulement les adresses ou les ports, mais le contenu textuel des

en-têtes HTTP afin de bloquer les scanners de vulnérabilités automatisés (ex. Nikto) ou les outils de reconnaissance applicative.

Scénario 4 : L'anomalie ICMP / Volume Supérieur à 1000 octets (Contournement & Évasion)

Justification : Trop souvent négligé, le protocole ICMP (ping) peut être détourné pour exfiltrer des données discrètement ou créer des canaux cachés (*tunneling*). Restreindre et auditer la taille des données (dsize:>1000) garantit la détection des anomalies de protocole hors du trafic web/ssh classique.

Scénario 5 : L'injection SQL Web (Exploitation de Faille - Critique / OSI 7)

Justification : Classée parmi les menaces les plus critiques du top OWASP, l'injection SQL menace directement l'intégrité et la confidentialité des bases de données. Ce scénario applique une alerte de Priorité 1 pour valider la protection des applications web contre les requêtes manipulant les flux de données.

Collecte et gestion des logs prioritaires en sécurité en relation avec les 5 scénarios avec justification.

Dans le cadre d'un centre de opérations de sécurité (SOC) ou d'une infrastructure d'entreprise, la centralisation et la gestion intelligente des logs sont fondamentales. L'accumulation brute de données réseau et système crée du "bruit" informationnel. Il est donc indispensable de classifier, prioriser et corrélérer les logs pour réagir efficacement aux incidents.

Notre architecture s'appuie sur un pipeline robuste : **Snort / Système OS → Syslog-ng → Filebeat → Elasticsearch → Kibana.**

Voici la matrice de gestion et de priorisation des logs mise en œuvre à travers nos 5 scénarios de test.

Matrice de priorisation et justification des logs par scénario

l. Scénario 1 : Scan de ports (Xmas Scan via Nmap) :

- **Logs prioritaires collectés :** * snort.log / Alertes Syslog contenant la signature BAD-TRAFFIC same SRC/DST ou indicateurs de paquets invalides (Flags FIN, PSH, URG activés simultanément).
- **Niveau de priorité : Moyenne (Faible à l'échelle d'un périmètre public, mais suspecte en zone interne).**
- **Justification de la collecte & Gestion :** Le scan Xmas est une technique de reconnaissance furtive conçue pour contourner les pare-feux sans état. Collecter ce log est indispensable, car il représente la **phase de reconnaissance** (première étape de la *Cyber Kill Chain*). Bien qu'il ne s'agisse pas d'une intrusion directe, l'indexation de ce log permet de détecter une cartographie active du réseau et d'ajouter préventivement l'IP source à une liste de surveillance ou de blocage temporaire avant qu'une attaque ciblée ne soit lancée.

m. Scénario 2 : Force brute SSH :

- **Logs prioritaires collectés :**
 - **Log Réseau (IDS) :** Alertes de détection comportementale Snort (detection_filter de requêtes SYN sur le port 22).

- **Log Système (OS) :** Échecs d'authentification répétés enregistrés dans /var/log/auth.log (Failed password for utilisateur_inexistant).
- **Niveau de priorité : Haute (Urgent si les tentatives proviennent d'une IP externe).**
- **Justification de la collecte & Gestion :** Ce scénario illustre parfaitement la nécessité de la **corrélation de logs hybrides (Réseau + Système)**. L'alerte de Snort à elle seule indique un flux rapide, mais le log d'authentification système confirme si l'attaque a échoué ou réussi. Prioriser ce log permet de déclencher des scripts d'atténuation automatique (comme *Fail2Ban*) pour interdire immédiatement l'IP au niveau du pare-feu applicatif de la machine hôte dès que le SIEM constate un dépassement de seuil (ex: 5 échecs en 10 secondes).

n. Scénario 3 : Outil de scan applicatif / User-Agent suspect (Nikto)

- **Logs prioritaires collectés :**
 - Alertes de l'inspecteur HTTP de Snort ciblant le champ http_header contenant la chaîne "Nikto".
- **Niveau de priorité : Moyenne.**
- **Justification de la collecte & Gestion :** Les outils automatisés comme Nikto effectuent des milliers de requêtes à la minute pour découvrir des répertoires cachés ou des vulnérabilités web. Capturer le *User-*

Agent permet d'identifier immédiatement le type d'outillage utilisé par l'attaquant. La gestion prioritaire de ce log au sein de Kibana permet d'éliminer le bruit de fond généré par les bots Internet légitimes (comme Googlebot) et d'isoler les outils de piratage afin de bloquer les adresses IP associées au niveau de la passerelle d'accès internet (WAF - *Web Application Firewall*).

o. Scénario 4 : Anomalie de protocole et volume ICMP (Ping volumineux)

- **Logs prioritaires collectés :**
 - Alertes Snort de type ICMP (itype:8) combinées avec l'argument de taille applicative dsize:>1000.
- **Niveau de priorité : Faute à Moyenne (Selon le contexte de l'infrastructure).**
- **Justification de la collecte & Gestion :** Le protocole ICMP (ping) est traditionnellement toléré et peu surveillé. Cependant, des paquets de ping transportant une charge utile supérieure à 1000 octets traduisent soit une tentative de déni de service par saturation (*Ping Flood*), soit une exfiltration de données masquée à l'intérieur du protocole (**Tunneling ICMP**). Collecter et prioriser ce log permet à l'équipe de sécurité de détecter des canaux d'exfiltration clandestins qui contourneraient les règles de pare-feu standards basées uniquement sur le trafic HTTP ou SSH.

p. Scénario 5 : Injection SQL (SQLi applicative)

- **Logs prioritaires collectés :**
 - Alertes Snort applicatives ciblant les signatures de requêtes textuelles de type "**UNION SELECT**" ou "**OR 1=1**".
- **Niveau de priorité : Critique / Priorité 1.**
- **Justification de la collecte & Gestion :** L'injection SQL cible directement l'accès non autorisé aux bases de données stockant des informations sensibles (identifiants, données financières, données personnelles). C'est une menace classée au sommet du top OWASP. Ce type de log doit impérativement être traité avec la plus haute priorité au sein du SIEM : une alerte de ce type doit immédiatement envoyer une notification en temps réel (via Webhook, SMS ou Email) aux analystes du SOC, car le risque de compromission de l'intégrité et de la confidentialité des données est immédiat et maximal.

Conclusion sur la stratégie de gestion du SIEM

La collecte de ces logs au sein d'une interface unique (Kibana) permet de passer d'une **sécurité réactive** (analyser les fichiers après un piratage) à une **sécurité proactive** (corrélérer les événements en temps réel). En appliquant des règles de priorisation précises (de *Moyenne* pour un scan de ports à *Critique* pour une injection SQL), l'infrastructure protège ses ressources de calcul et évite l'épuisement des analystes face aux faux positifs, garantissant ainsi une résilience optimale face aux cybermenaces actuelles.

Visualisation des logs et analyse approfondie des menaces

Pour assurer la résilience d'une infrastructure, l'analyse d'une alerte ne doit pas se limiter à sa simple détection. Elle nécessite une compréhension fine du mode opératoire de l'attaquant (*Playbook*) et une visualisation centralisée permettant de valider l'impact réel sur le système.

q. Scénario 1 : Le Scan de Ports (Xmas Scan)

- **Visualisation et Corrélation** : Dans Kibana, l'anomalie se traduit par une rafale de documents indexés en quelques secondes. Les logs affichent la signature réseau Snort **BAD-TRAFFIC same SRC/DST** ou des alertes liées à des paquets TCP invalides. L'analyse de l'histogramme montre un pic d'activité réseau très ciblé temporelle.
- **Explication de la menace** : Le scan Xmas (où les drapeaux FIN, PSH, et URG sont activés simultanément) est une technique de **reconnaissance furtive**. La menace réside dans le fait que l'attaquant cherche à cartographier les ports ouverts et à identifier le système d'exploitation sous-jacent en contournant les pare-feux traditionnels sans état (*stateless*). C'est le précurseur d'une attaque ciblée.

r. Scénario 2 : L'Attaque par Force Brute (SSH)

- **Visualisation et Corrélation** : La visualisation dans Kibana permet de corréler deux sources distinctes : l'alerte réseau de Snort (détection d'une fréquence anormale de paquets

SYN sur le port 22) et les logs d'authentification de l'OS transmis depuis **/var/log/auth.log (Failed password for utilisateur_inexistant)**. L'interface montre une explosion du nombre de documents générés en un intervalle de temps très court.

- **Explication de la menace :** La force brute SSH représente une menace directe de **compromission d'accès et d'élévation de privilèges**. Un attaquant utilisant des scripts automatisés tente de deviner les identifiants pour prendre le contrôle à distance de la machine. Si l'attaque réussit, l'attaquant obtient un shell sur le serveur, lui permettant de déployer des rançongiciels ou de pivoter à l'intérieur du réseau privé.

s. Scénario 3 : L'Outil de Scan Applicatif (User-Agent Suspect)

- **Visualisation et Corrélation :** L'alerte remonte dans le tableau de bord sous la règle personnalisée visant le champ applicatif **http_header** contenant la chaîne "**Nikto**". Même si le terminal de l'attaquant reçoit un **Connection refused** (port 80 fermé), la charge utile du paquet réseau est interceptée au vol et indexée instantanément dans Elasticsearch.
- **Explication de la menace :** L'utilisation d'un scanner automatisé comme *Nikto* représente une menace de **recherche active de vulnérabilités applicatives (Web)**. Ces outils automatisés testent des milliers de requêtes à la minute pour découvrir des répertoires cachés, des scripts obsolètes ou des failles de configuration. L'analyse du *User-*

Agent permet au SOC d'identifier précisément l'outillage de l'agresseur pour adapter la réponse.

t. Scénario 4 : L'Anomalie de Protocole (Exfiltration ICMP)

- **Visualisation et Corrélation** : Sur le terminal de l'attaquant, on observe le succès de pings transportant des paquets volumineux de 1500 octets. Côté Kibana, l'indexation de ces logs se traduit par une augmentation du volume de données collectées, activée par la règle de taille **dsize:>1000**.
- **Explication de la menace** : Le protocole ICMP (ping) est souvent ignoré par les outils de surveillance classiques. La menace représentée ici est le **Tunneling ICMP / l'Exfiltration discrète de données**. Un attaquant ayant déjà compromis un serveur peut encapsuler des données confidentielles ou des commandes de contrôle (C2) à l'intérieur de paquets de ping pour contourner les pare-feux qui bloquent le trafic sortant HTTP/SSH standard.

u. Scénario 5 : L'Injection SQL (SQLi)

- **Visualisation et Corrélation** : En saisissant le filtre **SQL** dans Kibana, on isole précisément les alertes relatives à la signature de requêtes malveillantes (comme "**UNION SELECT**"). Le log réseau capture l'intégralité de la chaîne de caractères malveillante passée en paramètre GET via l'outil de simulation **curl**.
- **Explication de la menace** : L'injection SQL est une menace **critique (Priorité 1)** pour la **confidentialité et l'intégrité des**

données. L'attaquant cherche à contourner l'interface applicative pour envoyer des commandes directement à la base de données sous-jacente. Cela peut lui permettre de contourner l'authentification, de voler des tables de données sensibles (informations clients, mots de passe hachés) ou même d'altérer et détruire des données stockées.

Synthèse de la valeur ajoutée du SIEM

La force de cette infrastructure réside dans sa capacité à traduire des flux réseau bruts et invisibles en indicateurs visuels clairs. En centralisant ces alertes, l'analyste de sécurité peut prioriser ses actions : appliquer un blocage automatique d'IP pour une injection SQL (critique) tout en surveillant de près l'évolution d'un scan de ports (reconnaissance).

Alertes et notifications pour les anomalies détectées avec des explications à l'administrateur du système

Le système de notification s'appuie sur la journalisation en temps réel générée par les manipulations réseau et système. Dès qu'une règle personnalisée dans /var/log/snort/alert ou un événement dans /var/log/auth.log est détecté par **syslog-ng / Filebeat**, le moteur de notification évalue la gravité de l'événement pour router l'alerte à l'administrateur.

[Commande Attaquante] → [Interface Réseau / Logs] → [Règle Snort / OS] → [Notification Admin]

- En fonction du Scan de ports (Manipulation **nmap -sX**)
 - **Déclencheur lié à ta manipulation** : Exécution de la commande **sudo nmap -sX 127.0.0.1** (Xmas Scan).
 - **Alerte générée à l'administrateur** : > **[ALERTE SÉCURITÉ - PRIORITÉ MOYENNE] : Activité de reconnaissance réseau suspecte.**
 - **Message** : L'IDS Snort a intercepté une signature **BAD-TRAFFIC same SRC/DST** ou un paquet contenant les flags TCP FIN, PSH et URG activés simultanément.
 - **Source** : **127.0.0.1** (Boucle locale) / Interface réseau locale.
 - **Explication textuelle à l'administrateur** : Un outil de scan (Nmap) tente de cartographier furtivement l'état de nos ports (ouvert/fermé). Cette technique cherche à contourner les règles de filtrage de base en envoyant des combinaisons de paquets illégitimes.
 - **Action administrative requise** : Identifier la machine d'origine sur le réseau local. Si l'activité provient d'un hôte externe, bloquer immédiatement l'IP source au niveau de la table **iptables** ou du pare-feu périmétrique.
- En fonction de la Force Brute SSH (**Manipulation for i in {1..7}; do ssh...**)
 - **Déclencheur lié à ta manipulation** : Lancement de la boucle Bash forçant des connexions successives rapides

avec l'argument NumberOfPasswordPrompts=0 -o
ConnectTimeout=2 utilisateur_inexistant@172.26.167.240.

- **Alerte générée à l'administrateur :**

[ALERTE CRITIQUE - HAUTE PRIORITÉ] : Attaque par Force Brute en cours sur le service SSH.

- **Message :** Corrélation établie. Snort signale un volume anormal de paquets SYN sur le port 22 et /var/log/auth.log enregistre plusieurs échecs consécutifs (Permission denied (publickey,password)) pour un utilisateur inconnu.
 - **Source :** IP de l'attaquant vers la cible 172.26.167.240.
 - **Explication textuelle à l'administrateur :** Une tentative d'intrusion automatisée cherche à compromettre les identifiants système pour obtenir un accès Shell à distance sur la machine.
 - **Action administrative requise :** Valider que le compte ciblé n'a pas été compromis (aucun statut Accept password). Le SIEM doit interagir avec un script d'atténuation (comme *Fail2ban*) pour bannir l'IP attaquante pendant 24 heures et s'assurer que l'authentification par mot de passe root est désactivée.
- En fonction du User-Agent Suspect (**Manipulation curl -H "User-Agent: Nikto"**)

- **Déclencheur lié à ta manipulation :** Requête web forcée avec l'en-tête modifié : **curl -H "User-Agent: Nikto" http://172.26.167.240/.**
- **Alerte générée à l'administrateur :**

[ALERTE SÉCURITÉ WEB - PRIORITÉ MOYENNE] : Utilisation d'un scanner de vulnérabilités applicatif.

- **Message** : Snort a détecté la chaîne de caractères "**Nikto**" au sein de l'en-tête HTTP (**http_header**) lors de l'inspection de paquets en profondeur.
- **Statut système** : Bien que la manipulation ait retourné un **Connection refused** (port 80 fermé), la signature malveillante a transité et a été bloquée à la source.
- **Explication textuelle à l'administrateur** : Un attaquant utilise un scanner automatisé pour identifier des failles de configuration web, des répertoires sensibles exposés ou des scripts obsolètes sur notre serveur.
- **Action administrative requise** : Conserver l'IP source sous surveillance renforcée. Si le serveur web (Apache/Nginx) venait à être activé sur ce port, s'assurer qu'un Pare-feu Applicatif Web (WAF) filtre les requêtes en amont.

➤ En fonction du Ping Volumineux (Manipulation **ping -s 1500**)

- **Déclencheur lié à ta manipulation** : Commande réseau forçant la taille de la charge utile : **ping -s 1500 -c 4 172.26.167.240**.
- **Alerte générée à l'administrateur** :

[ALERTE RÉSEAU - PRIORITÉ HAUTE] : Trafic ICMP anormal / Suspicion d'exfiltration.

- **Message** : Règle personnalisée déclenchée : Paquet ICMP Echo Request détecté avec un argument de taille **dsize:>1000**.
- **Volume observé** : Charge utile de 1500 octets par paquet transitant avec succès.
- **Explication textuelle à l'administrateur** : Le protocole de diagnostic ICMP (ping) est détourné de sa fonction initiale. Des paquets aussi volumineux masquent généralement une tentative d'exfiltration de données (*Tunneling ICMP*) ou une tentative de déni de service par saturation.
- **Action administrative requise** : Restreindre la taille des paquets ICMP acceptés au niveau du noyau Linux ou configurer le pare-feu pour rejeter les pings entrants en dehors des adresses IP de supervision internes autorisées.

➤ En fonction de l'Injection SQL (Manipulation **curl ... "UNION SELECT"**)

- **Déclencheur lié à ta manipulation** : Requête HTTP encodée contenant la charge utile SQL : **curl -G --data-urlencode "id=1 UNION SELECT null..." http://172.26.167.240/**.
- **Alerte générée à l'administrateur** :

[ALERTE ROUGE - PRIORITÉ MAXIMALE / CRITIQUE] : Tentative d'injection SQL active.

- **Message** : Correspondance exacte trouvée pour la signature applicative de niveau 7 : **"UNION SELECT"**.

L'événement est indexé en temps réel dans Elasticsearch sous la catégorie des menaces critiques.

- **Explication textuelle à l'administrateur :** Cette manipulation vise directement la base de données de l'entreprise. L'attaquant tente d'altérer la requête SQL initiale pour s'octroyer des privilèges administratifs, contourner l'authentification web ou dumper des données hautement confidentielles (mots de passe hachés, tables clients).
- **Action administrative requise d'urgence (SMS / Notification immédiate) :** Blacklister instantanément l'adresse IP source. Isoler les logs de l'application web pour s'assurer qu'aucune donnée n'a été retournée en réponse et obliger les développeurs à utiliser des requêtes préparées pour assainir les paramètres d'entrée.

CONCLUSION GÉNÉRALE

La réalisation de cet atelier a permis de concevoir, déployer et valider un système complet de détection d'anomalies et de gestion centralisée des logs (SIEM) combinant l'efficacité d'un IDS réseau (Snort) à la robustesse de la suite Elastic (ELK) et de syslog-ng. À travers la mise en œuvre de cinq scénarios d'attaque distincts — allant de la reconnaissance passive par scan Xmas jusqu'à l'exploitation applicative par injection SQL —, ce projet a mis en lumière l'importance cruciale de la visibilité en temps réel pour un administrateur système.

L'apport majeur de cette architecture réside dans sa capacité de corrélation hybride. Isoler des événements réseau ou des logs système de manière cloisonnée ne permet plus de contrer les menaces modernes. En centralisant des sources hétérogènes (comme les échecs d'authentification d'un OS et les alertes de flux de Snort), le pipeline déployé offre une vision contextuelle unique. De plus, la mise en place d'un mécanisme d'alerte ciblé et proportionnel à la criticité des attaques (Priorités 1 à 3) garantit une réponse aux incidents agile, limitant la fatigue des alertes pour l'équipe d'exploitation.

En conclusion, ce projet pose les bases d'une stratégie de défense en profondeur essentielle à la sécurisation des infrastructures industrielles et informatiques modernes. Les perspectives d'évolution de ce travail incluraient l'automatisation complète de l'atténuation (SOAR) par le biais de scripts de bannissement dynamique (IPS actif) dès la détection d'anomalies critiques sur le réseau.

GitHub: <https://github.com/Ear226/soc-ids-elk>

